

Lab #3: PostgreSQL, goose, and jmoiron/sqlx

Objectives

By the end of this lab, you should be able to

- access a PostgreSQL server using `psql`,
- use `goose` for database migrations, and
- interface a Go program with a database server.

Part 0: Overview of the lab

Listen to the lab introduction and watch the demo.

Part 1: Running a PostgreSQL server

In this part, you will create a running PostgreSQL server.

If you are running Linux without Docker or running Windows without a native PostgreSQL install, use a lab computer.

Ensure that you use the appropriate instructions below.

On your own computer (Docker)

On your own computer, obtain the Docker image for PostgreSQL by running the following command:

```
$ docker pull postgres
```

Once it is on your system, create a new Docker container with the following command:

```
$ docker run -ti -p 5432:5432 --name lab03 postgres
```

Notice the `--name lab03` argument in the command line – it provides the name of the resulting container.

On the lab computer

On the lab computer, you can launch a local PostgreSQL server instance using the script `start_postgres.sh` (available on Blackboard). Note the following about the resulting server:

- you will specify `/tmp` as the host when running `psql`, i.e.,
`psql -h /tmp -U postgres`
- you will include `host=/tmp` in the connect string when coding in Go

Part 2: Creating the database

Later in Part 3, you will use the command `goose` combined with files containing SQL statements as you create (and destroy) the database schema. Prior to running `goose`, you must create a database. In Part 2 (here), you will create a database using `psql`.

On your own computer (Docker)

When Docker is running in the container, it binds to a UNIX domain socket in addition to TCP port 5432.¹ This UNIX domain socket exists in the container file system, and with that being the case, if `psql` runs within the same container, it has access to this socket. Since the client `psql` was built with the same configuration as the server, it will automatically look for this socket.

The following command will run `psql` in the container named `lab03`:

```
$ docker exec -ti lab03 psql -U postgres
```

On the lab computer

When the PostgreSQL server is running on the lab machines, it binds to a UNIX domain socket in `/tmp` in addition to TCP port 5432. The following command will run `psql` and connect to the UNIX domain socket in `/tmp`:

```
$ psql -h /tmp -U postgres
```

Creating database lab03

Within `psql`, run the following command to create the database:

```
CREATE DATABASE lab03;
```

Enter the command `\q` to terminate `psql`.

Part 3: Creating the schema with goose

1. Download `goose` with the following command:

```
$ go get -u github.com/pressly/goose/cmd/goose
```

This command will be added to your `go/bin` directory.
2. Within your `lab03` directory, create a directory named `migrations`. Change into this new directory.
3. Inside the directory `migrations`, run the following command line to create a skeleton migration file:

```
$ goose postgres "user=postgres dbname=lab03 ssl=disable" create init sql
```

You may need to specify the path to `goose`. Moreover, you may need to add the string `host=localhost` or `host=/tmp` to the connect string above.

¹In the Docker container, PostgreSQL has been configured to bind to `/run/postgresql/.s.PGSQL.5432`.

4. Open the created file `2019*_init.sql`. Within this file, you need to complete two sections:

(a) Following `-- +goose Up`, provide the SQL statements that create the tables.

(b) Following `-- +goose Down`, provide the SQL statement that destroy the tables.

Familiarize yourself with the following Go structures. Create a database schema that mirrors these structures.

```
type student struct {
    StudentId int    `db:"student_id"`
    FirstName  string `db:"first_name"`
    LastName   string `db:"last_name"`
}

type course struct {
    CourseId      int    `db:"course_id"`
    Department    string `db:"department"`
    CourseNumber  int    `db:"course_number"`
    Section       string `db:"section"`
}

type enrollment struct {
    EnrollmentId int `db:"enrollment_id"`
    StudentId    int `db:"student_id"`
    CourseId     int `db:"course_id"`
}
```

5. After writing the SQL statements that create (and destroy) the three tables, you need to use `goose` to upgrade your (empty) database to incorporate these tables.

```
$ goose postgres "user=postgres dbname=lab03 ssl=disable" up
```

Part 4: github.com/jmoiron/sqlx

1. Review the file `lab03.go`, which is available on Blackboard. Ensure that you understand everything in this file.
2. Attempt to build a binary from the unmodified `lab03.go`.

```
$ go build
```

You may need to download missing packages, e.g.,

```
$ go get github.com/jmoiron/sqlx
$ go get github.com/lib/pq
```

3. Once `lab03` builds, try running it. It should terminate with an error:

```
cannot connect to database: must write code to connect to the database
```

4. In `lab03.go`, implement the function `connectToDB`. Verify your implementation by ensuring that the previous error disappears.

5. In `lab03.go`, implement the unimplemented functions.

The create functions should produce output like what follows:

```
$ ./lab03 createStudent Jane Doe
1 student(s) created.
```

The list functions should produce output like what follows:

```
$ ./lab03 listStudents
ID    Last name, first name
-----
1     Doe, Jane
2     Smith, John
```